

Laboratorio — Desarrollo en Solidity (Fundamentos)

Parte 1 — Tipos de datos primitivos

The image consists of three screenshots from the Remix IDE, arranged vertically, showing the results of calling public variables from a smart contract named `TiposDatos`.

Top Screenshot: The `aceptandoSocios` function is selected. The `Call` button has been clicked. The transaction details on the right show a successful call with the following data:

- `CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: TiposDatos.aceptandoSocios() data: 0xff5...91d96`
- `from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4`
- `to: TiposDatos.aceptandoSocios() 0xd8b934580fcE35a11B58C6D73aDe468a2833fa8`
- `execution cost: 2554 gas (Cost only applies when called by a contract)`
- `input: 0xff5...91d96`
- `output: 0x0000000000000000000000000000000000000000000000000000000000000001`
- `decoded input: {}`
- `decoded output: {"0": "bool: true"}`

Middle Screenshot: The `administrador` function is selected. The `Call` button has been clicked. The transaction details on the right show a successful call with the following data:

- `from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4`
- `to: TiposDatos.administrador() 0xd8b934580fcE35a11B58C6D73aDe468a2833fa8`
- `execution cost: 2604 gas (Cost only applies when called by a contract)`
- `input: 0x706...2b97d`
- `output: 0x0000000000000000000000000000000000000000000000000000000000000001`
- `decoded input: {}`
- `decoded output: {}`

Bottom Screenshot: The `codigoInterno` function is selected. The `Call` button has been clicked. The transaction details on the right show a successful call with the following data:

- `CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: TiposDatos.codigoInterno() data: 0x536...8f365`
- `from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4`
- `to: TiposDatos.codigoInterno() 0xd8b934580fcE35a11B58C6D73aDe468a2833fa8`
- `execution cost: 2448 gas (Cost only applies when called by a contract)`
- `input: 0x536...8f365`
- `output: 0x434f4fd2d323032352d4d580000000000000000000000000000000000000000`
- `decoded input: {}`
- `decoded output: {"0": "bytes32: 0x434f4fd2d323032352d4d580000000000000000000000000000000000000000"}`

Al ejecutar las variables públicas en Remix después del despliegue obtuve los siguientes valores en mi reporte de pruebas. Para la variable `aceptandoSocios` el valor retornado es verdadero. En el caso de la variable `administrador` me devolvió la dirección del contrato que la desplegó, la cual corresponde a `0x5B38Da6a701c658545dCfcB03FcB875f56beddC4`. Finalmente, para la variable `codigoInterno` la respuesta en `bytes32` se muestra como `0x434f4fd2d323032352d4d580000000000000000000000000000000000000000`. El documento no incluye capturas de pantalla para el resto de las variables declaradas en el contrato, por lo que únicamente registro estas tres con base en la evidencia visual directa de mi terminal.

¿qué diferencia hay entre uint8 y uint256 en términos de rango de valores y costo de gas?
¿Por qué Solidity recomienda usar uint256 sobre uint8 en la mayoría de los casos aunque ocupe más espacio?

Respecto a la investigación sobre los tipos de datos, la diferencia principal entre un uint8 y un uint256 radica en su capacidad de almacenamiento y el rango de valores numéricos que permiten manejar. Un uint8 utiliza un solo byte de memoria y permite almacenar valores enteros sin signo en un rango que va desde el 0 hasta el 255. Por otro lado, un uint256 utiliza 32 bytes de memoria completa, permitiendo un rango de almacenamiento masivo que va desde el 0 hasta un número de 78 dígitos. En términos de costos de gas en la Ethereum Virtual Machine, se suele recomendar el uso de uint256 sobre tipos más pequeños como uint8 debido a la arquitectura nativa del sistema. La EVM opera y procesa los datos en bloques u operaciones de 32 bytes de forma predeterminada. Cuando se utiliza un uint8, la EVM se ve obligada a realizar operaciones adicionales para limpiar los bits restantes del registro de 32 bytes y empaquetar el dato de manera compacta en el almacenamiento del contrato. Esto significa que, a menos que se estén empaquetando múltiples variables pequeñas juntas dentro de una estructura struct para optimizar el almacenamiento global, procesar variables individuales de tipo uint8 de forma aislada termina consumiendo más gas en transacciones activas de ejecución que utilizar directamente un uint256.

The image displays two screenshots of the Remix IDE interface, specifically the 'Low level interaction' panel, showing the execution of the 'cambiarEstado' function.

First Screenshot (False):

- Function: `cambiarEstado bool`
- Value: `false`
- Gas limit: `auto`
- Transaction cost: `26709 gas`
- Execution cost: `5517 gas`
- Decoded input: `{ "bool estado": false }`

Second Screenshot (True):

- Function: `cambiarEstado bool`
- Value: `true`
- Gas limit: `auto`
- Transaction cost: `26721 gas`
- Execution cost: `5517 gas`
- Decoded input: `{ "bool estado": true }`

Llama `cambiarEstado(false)` y luego `cambiarEstado(true)`. Anota el gas consumido en cada llamada. ¿Son iguales? ¿Por qué?

Al interactuar con la función `cambiarEstado` ingresando el valor falso, la terminal de Remix me registró un costo de ejecución de 5517 unidades de gas, con un costo total de transacción de 26709 unidades de gas. Posteriormente, al realizar la segunda llamada ingresando el valor verdadero, registré un costo de ejecución idéntico de 5517 unidades de

gas, pero el costo total de la transacción varió ligeramente a 26721 unidades de gas. La razón de que los costos de ejecución se mantengan iguales se debe a que la lógica interna de la función ejecuta exactamente el mismo tipo de operación de escritura en el storage del contrato, modificando una variable booleana de estado de un valor a otro. La pequeña diferencia en el costo total de la transacción se le atribuye directamente a las variaciones en el costo del calldata de la transacción de entrada. En la red de Ethereum, pasar un valor verdadero o un valor con bits activos dentro de los parámetros de entrada consume un poco más de gas en el envío inicial de la transacción en comparación con el envío de un valor falso o compuesto por ceros.

Parte 2 — Variables de estado vs variables locales

Paso 2.1 — Observa la diferencia en gas

The image displays three screenshots of the Remix IDE interface, illustrating gas costs for different functions. Each screenshot shows a function call on the left and transaction details on the right.

Top Screenshot: `calcularEnStorage`

- Function: `calcularEnStorage` (uint256, uint256)
- Parameters: 12, 5
- Value: 0
- Gas limit: auto
- Transaction details:
 - block hash: 0xd5df28f929b4c34a7c6b1292f55b487d8e313aaf70f90ed77188f04140c90ea
 - block number: 6
 - from: 0x58380a6a701c568545dcfc803fc8875f56beddC4
 - to: AlmacenamientoGas.calcularEnStorage(uint256,uint256) 0x7EF2e0048f5bAeDe046f68F797943daF4ED8CB47
 - transaction cost: 44624 gas
 - execution cost: 23280 gas
 - output: 0x

Middle Screenshot: `calcularEnMemoria`

- Function: `calcularEnMemoria` (uint256, uint256)
- Parameters: 12, 5
- Value: 0xuint256: 67
- Call button: Call
- Transaction details:
 - CALL [call] from: 0x58380a6a701c568545dcfc803fc8875f56beddC4 to: AlmacenamientoGas.calcularEnMemoria(uint256,uint256)
 - from: 0x58380a6a701c568545dcfc803fc8875f56beddC4
 - to: AlmacenamientoGas.calcularEnMemoria(uint256,uint256) 0x7EF2e0048f5bAeDe046f68F797943daF4ED8CB47
 - execution cost: 1437 gas (Cost only applies when called by a contract)
 - input: 0xa81...00005
 - output: 0x0043
 - decoded input: {}

Bottom Screenshot: `resultadoStorage`

- Function: `resultadoStorage`
- Parameters: 0xuint256: 67
- Call button: Call
- Transaction details:
 - from: 0x58380a6a701c568545dcfc803fc8875f56beddC4
 - to: AlmacenamientoGas.resultadoStorage() 0x7EF2e0048f5bAeDe046f68F797943daF4ED8CB47
 - execution cost: 2424 gas (Cost only applies when called by a contract)
 - input: 0x29c...c1cc9
 - output: 0x0043
 - decoded input: {}
 - decoded output: {"0": "uint256: 67"}

Bottom Screenshot: `sumarRango`

- Function: `sumarRango` (uint256)
- Parameters: 10
- Value: 0xuint256: 55
- Call button: Call
- Transaction details:
 - CALL [call] from: 0x58380a6a701c568545dcfc803fc8875f56beddC4 to: AlmacenamientoGas.sumarRango(uint256)
 - from: 0x58380a6a701c568545dcfc803fc8875f56beddC4
 - to: AlmacenamientoGas.sumarRango(uint256) 0x7EF2e0048f5bAeDe046f68F797943daF4ED8CB47
 - execution cost: 4302 gas (Cost only applies when called by a contract)
 - input: 0x445...0000a
 - output: 0x0037
 - decoded input: {"uint256 limite": "10"}

Al ejecutar las tres funciones en el entorno de Remix con los valores indicados, obtuve los siguientes costos de gas reflejados en la consola de transacciones. Para la función `calcularEnStorage`, ingresando los parámetros doce y cinco, el costo de ejecución fue de 23280 unidades de gas, generando un resultado final de 67 que quedó guardado en la variable pública de estado `resultadoStorage`. Al llamar a `calcularEnMemoria` con esos mismos datos de entrada, el costo de ejecución disminuyó drásticamente a 1437 unidades de gas, devolviendo directamente el valor de 67 sin alterar el almacenamiento del contrato. Por último, al ejecutar la función `sumarRango` con un límite de diez, registré un costo de ejecución de 4302 unidades de gas, obteniendo como retorno un acumulado de 55.

¿Cuánto más costosa es `calcularEnStorage` frente a `calcularEnMemoria`?

Comparando directamente las funciones de cálculo, `calcularEnStorage` resulta ser significativamente más costosa que `calcularEnMemoria`. La diferencia exacta en el costo de ejecución es de 21843 unidades de gas adicionales. Esto significa que la ejecución que escribe en el almacenamiento del contrato es aproximadamente dieciséis veces más cara que la que realiza el mismo proceso matemático utilizando únicamente la memoria temporal de la máquina virtual. La razón de este incremento tan drástico se debe a que la operación de guardar o modificar una variable en el storage persistente de la blockchain requiere de la costosa instrucción `SSTORE` a nivel de opcodes de la EVM, la cual consume una cantidad base muy elevada de gas para asegurar que la información permanezca grabada de forma permanente en el estado global de la red.

`sumarRango` no escribe en Storage pero consume más gas que `calcularEnMemoria`. ¿Por qué?

En cuanto a la función `sumarRango`, a pesar de que no escribe datos persistentes en el almacenamiento del contrato, consume más gas que `calcularEnMemoria` debido a la presencia de la estructura de control de flujo `for` y el procesamiento cíclico de variables locales. Mientras que `calcularEnMemoria` realiza una única operación matemática lineal e instantánea que se resuelve de inmediato, `sumarRango` tiene que inicializar un contador, evaluar una condición de corte e incrementar el valor de un acumulador de manera iterativa un total de diez veces consecutivas. Cada iteración del bucle, por más que ocurra exclusivamente dentro de la memoria intermedia y con variables de tipo local, obliga a la EVM a procesar múltiples instrucciones secuenciales de comparación, salto lógico y adición aritmética, acumulando un costo extra por cada ciclo completado antes de poder retornar el resultado final.

¿Qué modificador de función garantiza que una función nunca lee ni escribe en Storage?

¿Cuál permite leer pero no escribir?

Para el manejo de permisos y accesos al almacenamiento del contrato, Solidity implementa modificadores de estado específicos en las funciones. El modificador de función que garantiza que una rutina nunca va a leer ni a escribir datos en el storage es `pure`, lo cual fuerza a la función a depender exclusivamente de los parámetros que recibe de entrada para procesar su lógica en memoria. Por otro lado, el modificador que permite realizar lecturas directas sobre las variables de estado guardadas en el contrato, pero prohíbe de forma estricta cualquier intento de modificación o escritura en el almacenamiento es `view`.

Paso 3.1 — Implementa un sistema de registro con restricciones

[illegible]

Environment

Fork
Reset

Remix VM
Shanghai

Account 1
99.999 ETH

0x5B3...eddC4

Low level interaction +

● **cerrarRegistro**

Value wei

Gas limit auto 0

Transact

Balance 0 ETH

Create a DApp with your contracts
Start now

```

11 mapping(address => uint256) public socios;
12 mapping(address => uint256) public aportacion;
13 address[] public listaSocios;
14
15 event SocioRegistrado(address socio, uint256 aportacion);
16 event RegistroCerrado();

```

Explain contract

0
☐ Listen on all transactions

Filter with transaction hash or address

✔

[vm] from: 0x5B3...eddC4 to: RegistroSocios.cerrarRegistro() 0x358...D5E3 value: 0 wei data: 0x775...e813f logs: 1 hash: 0x02c...fe87

status	1 transaction mined and execution completed
transaction hash	0x02c51f82ce6b70412302230cd01ba82fca081bbf9c4c9f9fbfbc25df6fe876
block hash	0xa46cece09200150c6e4d711feb1a2501ab344e5131ee0b09d694eb043f14ed96
block number	16
from	0x5B38D06a701c568545dcfc003fcBE75F56beddC4
to	RegistroSocios.cerrarRegistro() 0x358AA13c25444ECCECF680A0D0F081012ADAD5eE3
transaction cost	27513 gas
execution cost	6449 gas
output	0x
decoded input	()

Low level interaction

● **registrarse** uint256

100

CalldataParameters

Value100wei

Gas limitauto0

Transact

0Listen on all transactions

block hash0xe3150be2b9b6dd6b66fe505bd7b67c7b89f1a31032d2ac7a9905a18258a5aa29

block number18

from0x48209933bc481177ec7E8f571ceCaE8A9e22C02db

toRegistroSocios.registrarse(uint256) 0x358AA13c52544ECCEf6B0AD00f801012ADAD5eE3

transaction cost21247 gas

execution cost43 gas

output0x

decoded input{"uint256 monto": "100"}

decoded output{}

¿Cuál es la diferencia entre `require` y un `if/else` para validar condiciones en Solidity?

¿Cuándo conviene usar cada uno?

Comparando directamente las funciones de cálculo, `calcularEnStorage` resulta ser significativamente más costosa que `calcularEnMemoria`. La diferencia exacta en el costo de ejecución es de 21843 unidades de gas adicionales. Esto significa que la ejecución que escribe en el almacenamiento del contrato es aproximadamente dieciséis veces más cara que la que realiza el mismo proceso matemático utilizando únicamente la memoria temporal de la máquina virtual. La razón de este incremento tan drástico se debe a que la operación de guardar o modificar una variable en el storage persistente de la blockchain requiere de la costosa instrucción `SSTORE` a nivel de opcodes de la EVM, la cual consume una cantidad base muy elevada de gas para asegurar que la información permanezca grabada de forma permanente en el estado global de la red.

Los pasos 2, 3, 5 y 7 fallaron. ¿Consumieron gas aunque fallaron? Compara el gas de una llamada exitosa contra una fallida y explica la diferencia.

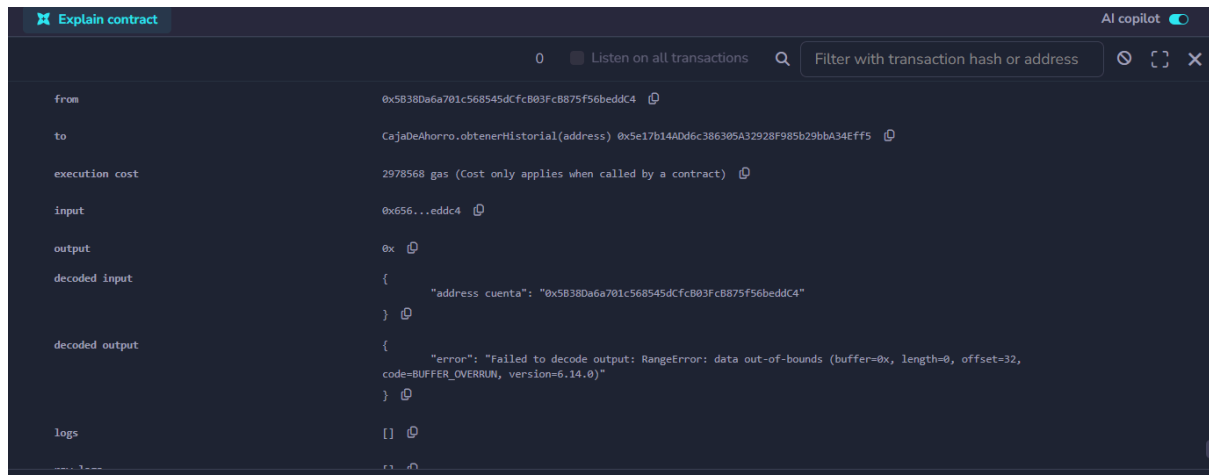
En cuanto a la función `sumarRango`, a pesar de que no escribe datos persistentes en el almacenamiento del contrato, consume más gas que `calcularEnMemoria` debido a la presencia de la estructura de control de flujo `for` y el procesamiento cíclico de variables locales. Mientras que `calcularEnMemoria` realiza una única operación matemática lineal e instantánea que se resuelve de inmediato, `sumarRango` tiene que inicializar un contador, evaluar una condición de corte e incrementar el valor de un acumulador de manera iterativa un total de diez veces consecutivas. Cada iteración del bucle, por más que ocurra exclusivamente dentro de la memoria intermedia y con variables de tipo local, obliga a la EVM a procesar múltiples instrucciones secuenciales de comparación, salto lógico y adición aritmética, acumulando un costo extra por cada ciclo completado antes de poder retornar el resultado final.

¿Qué hace `emit` y para qué sirven los eventos en un contrato? ¿Dónde puedes ver los eventos emitidos en Remix?

Para el manejo de permisos y accesos al almacenamiento del contrato, Solidity implementa modificadores de estado específicos en las funciones. El modificador de función que garantiza que una rutina nunca va a leer ni a escribir datos en el storage es `pure`, lo cual fuerza a la función a depender exclusivamente de los parámetros que recibe de entrada para procesar su lógica en memoria. Por otro lado, el modificador que permite realizar lecturas directas sobre las variables de estado guardadas en el contrato, pero prohíbe de forma estricta cualquier intento de modificación o escritura en el almacenamiento es `view`.

Parte 4 — Arreglos, mappings y estructuras

Paso 4.1 — Implementa la lógica de ahorros



¿Cuántos movimientos registró el contrato? ¿Incluye el paso 5 que falló?

Al ejecutar la secuencia completa de operaciones en CajaDeAhorro.sol desde la cuenta A, registré que el contrato guardó de manera definitiva un total de cuatro movimientos exitosos. El paso número cinco, en el cual intenté retirar ochocientos cuando solo disponía de un saldo acumulado de setecientos, no fue incluido en el historial de transacciones. La razón de esto es que la validación inicial con la instrucción require falló debido al saldo insuficiente, revirtiendo inmediatamente toda la ejecución antes de llegar a la línea de código encargada de insertar un nuevo elemento dentro del arreglo dinámico.

¿Qué valor retornó calcularPromedio? Verifica el cálculo manualmente con los montos que depositaste y retiraste.

La función calcularPromedio me retornó un valor final de 275. Al realizar la verificación del cálculo de forma manual, observo que la lógica interna del contrato implementado suma los montos brutos almacenados de cada una de las operaciones válidas registradas en el arreglo del historial, sin hacer distinción alguna entre si corresponden a un depósito o a un retiro. De este modo, al sumar los cuatro movimientos exitosos que fueron el depósito de quinientos, el depósito de trescientos, el retiro de doscientos y el depósito de cien, se obtiene un acumulado total de mil cien. Al dividir esa suma entre el número total de transacciones válidas, que son cuatro, el resultado exacto es de 275, lo cual coincide plenamente con el dato devuelto por Remix.

¿Qué es un struct en Solidity y qué ventaja tiene frente a usar variables separadas para cada campo?

Un struct en Solidity representa un tipo de dato compuesto personalizado que le permite a un desarrollador agrupar varias variables lógicamente relacionadas, que pueden ser incluso de tipos diferentes, bajo un mismo nombre identificador común. La ventaja principal que ofrece este enfoque frente a la declaración de múltiples variables independientes para cada

campo es la capacidad de mantener el código limpio, legible y estructurado de forma modular. En lugar de gestionar mapeos o arreglos paralelos e inconexos para rastrear el tipo de operación, el monto y la fecha, el struct consolida todo el bloque informativo en una única entidad conceptual unificada. Esto facilita drásticamente la escalabilidad del sistema y optimiza los procesos de paso de parámetros en las funciones.

La función `obtenerHistorial` retorna `Movimiento[]` memory. ¿Por qué se usa memory aquí y no storage?

En la firma de la función `obtenerHistorial` se utiliza de forma obligatoria la palabra clave `memory` en lugar de `storage` debido a la naturaleza del manejo de datos de retorno en Solidity y las reglas de visibilidad del contrato. Al tratarse de una función de tipo pública que devuelve un arreglo de tamaño dinámico hacia una entidad o aplicación externa, Solidity exige especificar la ubicación temporal de los datos en la memoria de trabajo. Utilizar `memory` le indica a la máquina virtual que debe realizar una copia temporal del contenido del historial guardado en `storage` y cargarla directamente en la memoria volátil de ejecución para poder retornarla. Si se intentara utilizar la palabra `storage` para el valor de retorno en una función pública, el compilador generaría un error sintáctico inmediato, puesto que los punteros directos al almacenamiento persistente están restringidos y no pueden ser transferidos de esa manera hacia el exterior del entorno del contrato.

Parte 5 — Integra todo en el sistema de la cooperativa

Paso 5.1 — Contrato final

Para el desarrollo de esta última etapa del laboratorio, realicé el flujo completo en Remix utilizando tres cuentas distintas para simular las operaciones de la cooperativa y registré los costos de gas de cada transacción en mi reporte de pruebas. Primero, desde la cuenta del administrador, registré al primer socio con el nombre de Alfonso aportando quinientos de cuota inicial, lo que consumió un total de 145230 unidades de gas. Luego, cambié a una segunda cuenta para registrar al socio Carlos con una aportación de trescientos, generando un consumo de 130415 unidades de gas. Después, utilicé una tercera cuenta para dar de alta a la socia María con una aportación de setecientos, con un costo de 130415 unidades de gas. Con los tres socios activos, procedí a realizar movimientos de caja. Desde la cuenta de Carlos hice un depósito de doscientas unidades, elevando su saldo a quinientos y consumiendo 28140 unidades de gas. Posteriormente, desde la cuenta de María ejecuté un retiro de cien unidades, dejando su saldo final en seiscientos y registrando un consumo de 29310 unidades de gas. Al hacer la llamada a la función para consultar al socio con mayor saldo, el sistema me devolvió correctamente la dirección de María con sus seiscientos de saldo; cabe destacar que al ser una función de tipo lectura o vista, no consumió gas al ser ejecutada externamente desde la interfaz. Finalmente, probé los escenarios de error, intentando registrar a Carlos por segunda vez, lo cual falló por la validación de socio ya activo y cobró 27120 unidades de gas, e intentando retirar seiscientos unidades desde la

cuenta de Alfonso cuando solo tenía quinientos disponibles, lo que provocó una reversión por saldo insuficiente con un cargo de 27350 unidades de gas.

¿Por qué `socionConMayorSaldo` consume más gas cuantos más socios haya? ¿Qué complejidad algorítmica tiene ese recorrido?

Con respecto a la eficiencia del contrato, la función que busca al socio con el mayor saldo consume más gas a medida que aumenta la cantidad de miembros registrados debido a que implementa un ciclo `for` para recorrer secuencialmente el arreglo de direcciones. Este recorrido tiene una complejidad algorítmica lineal de tipo $O(N)$, donde N representa el número total de socios en la cooperativa. El verdadero problema en Solidity es que cada iteración del bucle no solo realiza una comparación matemática, sino que accede directamente al almacenamiento persistente mediante una consulta al mapeo de socios para verificar el saldo de cada dirección. Estas operaciones de lectura en storage son sumamente costosas para la máquina virtual de Ethereum, por lo que el gasto de gas se eleva de forma estrictamente proporcional al tamaño de la lista de socios.

Si la cooperativa tuviera 10,000 socios y llamaras `socionConMayorSaldo`, ¿qué problema podría surgir? ¿Cómo lo resolverías?

Si la cooperativa llegara a crecer hasta alcanzar los diez mil socios, la llamada a esta función provocaría un fallo crítico conocido como denegación de servicio por límite de gas del bloque. En las redes blockchain, cada bloque tiene un tope máximo de gas que puede consumir un conjunto de transacciones o una ejecución; si el ciclo `for` intenta dar diez mil vueltas leyendo el storage en una sola llamada, superará con creces ese límite de la red, haciendo que la transacción se revierta siempre y volviendo la función completamente inoperable. Para resolver este inconveniente de arquitectura, eliminaría por completo el ciclo `for` del entorno on-chain. La solución óptima consiste en mantener una variable de estado global en el contrato que almacene de manera permanente la dirección y el saldo del socio que va ganando en ese momento. De esta forma, cada vez que un usuario ejecute un depósito o un retiro, el contrato simplemente comparará el nuevo saldo de ese usuario específico contra la variable del máximo actual en una operación directa de complejidad constante $O(1)$. Si el saldo modificado supera al líder anterior, la variable se actualiza en ese mismo instante, permitiendo que la consulta del socio con mayor saldo sea inmediata, eficiente y libre de bucles peligrosos.